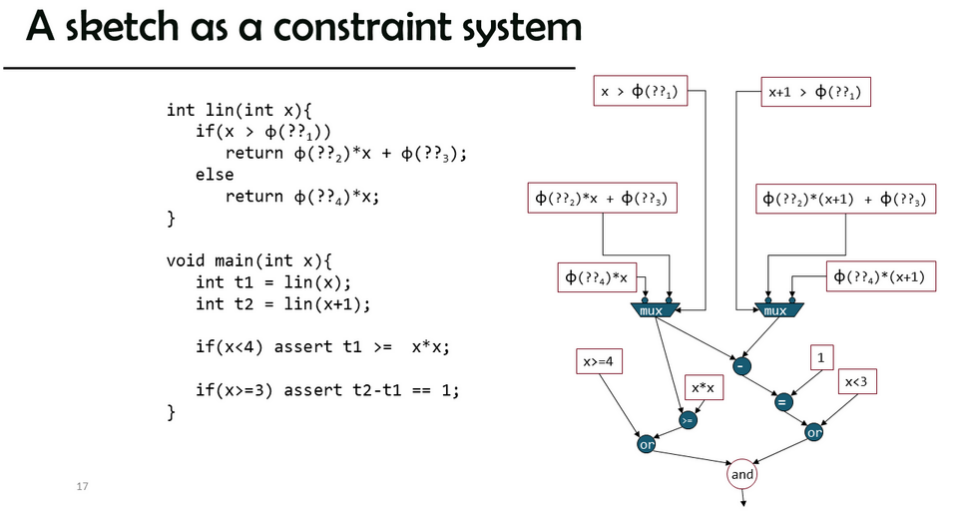
LECTURE 8

Solving constraints with SAT solvers

8.1 Representing Sets and Symbolic Expressions



The symbolic execution defined earlier relies on our ability to compactly represent both the symbolic values Ψ and the set of viable candidates Φ. For the symbolic values, the representation is simply as an AST with unknowns at the leaves. For the set Φ, we represent them as predicates. The idea of representing sets as predicates is very common in many different areas of program analysis and synthesis. The idea is to represent a set Φ as a predicate $P_\Phi(\phi)$ such that $P_\Phi(\phi)$ iff $\phi \in \Phi$. Thus, for example, the predicate $true$ corresponds to the universal set, and the standard operations of Union and intersection correspond to and and or of the corresponding predicates respectively.

From the semantics, for example, we see that assert restricts the set to those ϕ for which the expression is true. This means that if we initially had a set represented by a predicate $P_\Phi(\phi)$, then after the assert, the set would be represented by the new predicate

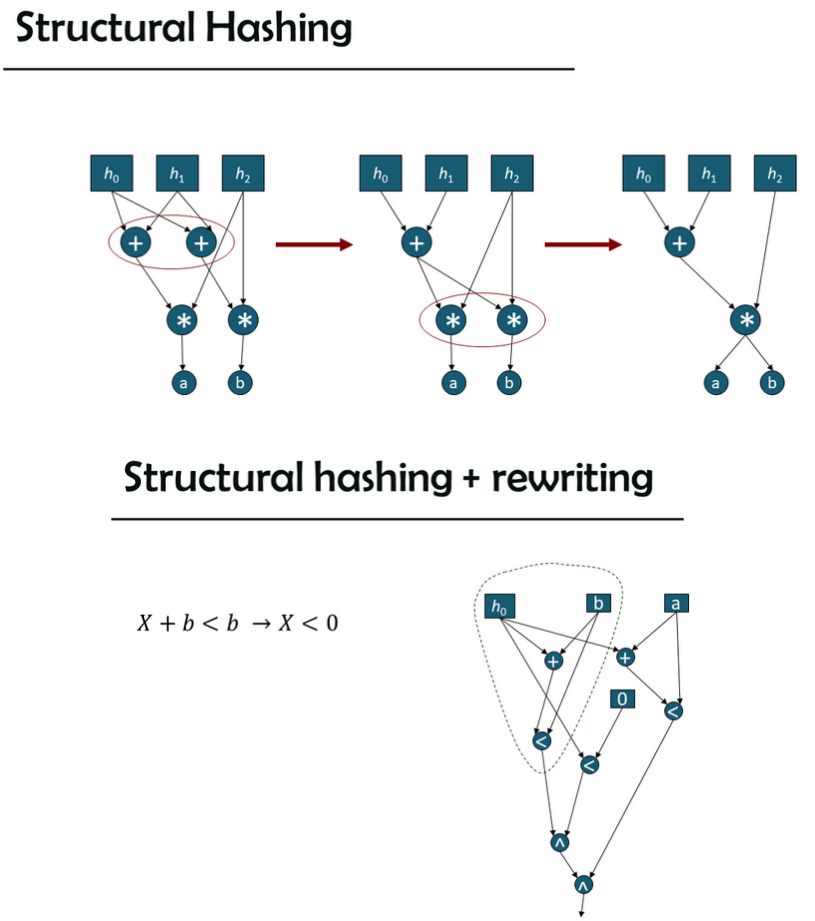
$$P_\Phi(\phi) \wedge f(\phi) = 1 \text{ where } f(\phi) = \mathcal{A}[\![e]\!]^\tau \sigma \phi$$

Similarly, the union in the semantics of `if` would be represented as

$$P_{\Phi_1}(\phi) \vee P_{\Phi_2}(\phi)$$

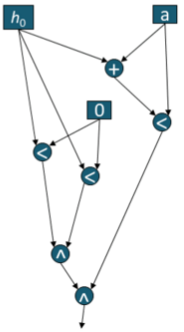
The figure above provides an example of the representation of the set of valid examples for a given sketch. The nodes labeled as `mux` simply select from their two inputs based on whether a condition is true or false. The `and` joins together the constraints from the two asserts, each of which involves an `or` because the asserts are guarded by `if` conditions. Another point to note is that the representation is not quite a tree, but a DAG. This is simply an optimization to exploit sharing in the underlying expression.

8.2 Optimizing the representation



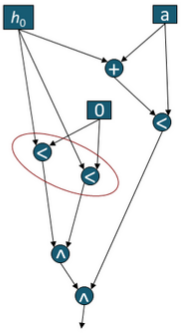
Structural hashing + rewriting

$X + b < b \rightarrow X < 0$



Structural hashing + rewriting

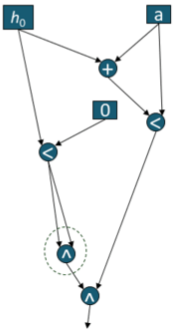
$X + b < b \rightarrow X < 0$



Structural hashing + rewriting

$X + b < b \rightarrow X < 0$

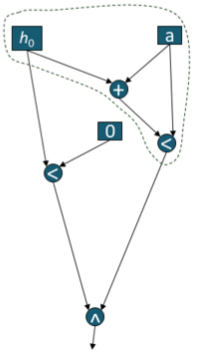
$X \wedge X \rightarrow X$



Structural hashing + rewriting

$X + b < b \rightarrow X < 0$

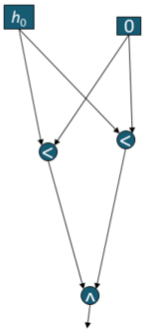
$X \wedge X \rightarrow X$



Structural hashing + rewriting

$X + b < b \rightarrow X < 0$

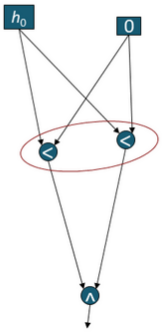
$X \wedge X \rightarrow X$

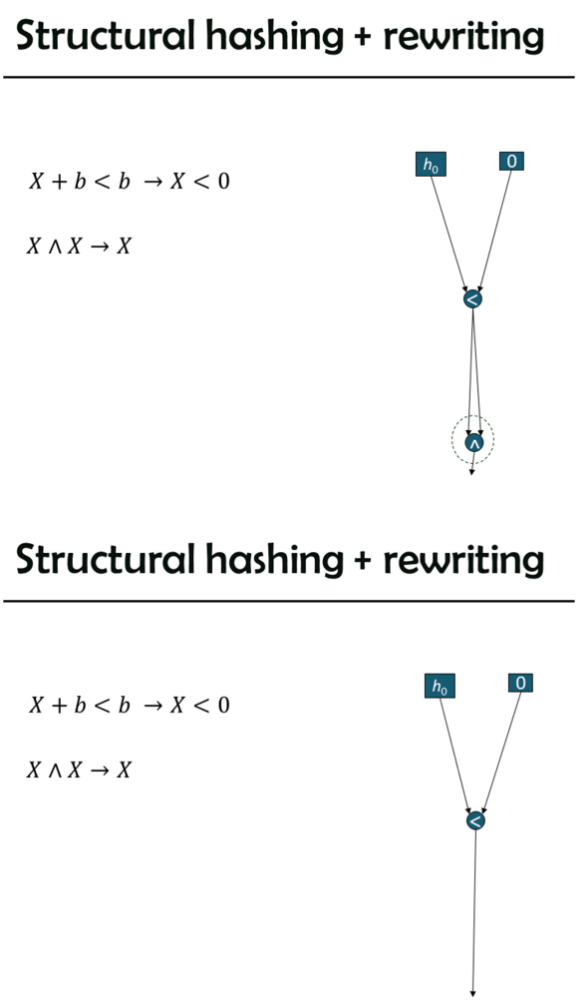


Structural hashing + rewriting

$X + b < b \rightarrow X < 0$

$X \wedge X \rightarrow X$





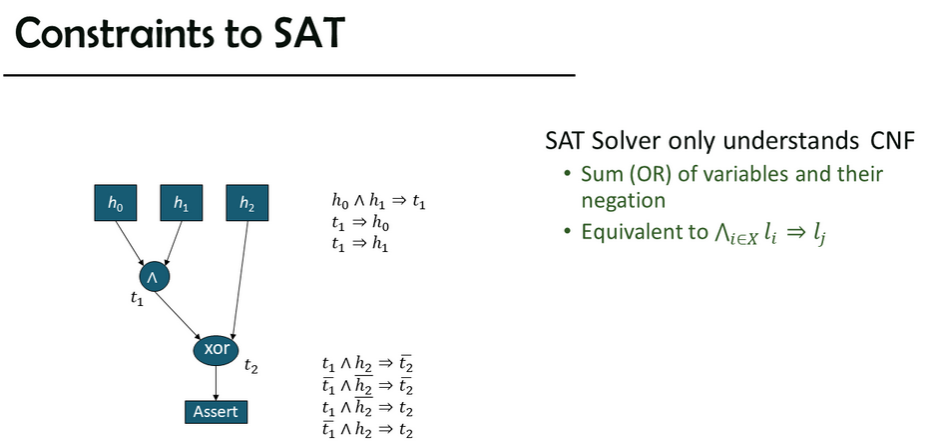
There are two major optimizations that are used to reduce the size of the representation: structural hashing and algebraic simplification. Structural hashing is illustrated by the animation. The idea is simply to identify common subexpressions and represent them with the same node. Because the representation is a DAG, it is sufficient to traverse it from the leaves down to the root node in one pass. For every node we record its type and the ID of its parents. If two nodes of the same type share the same parents, they get merged into a single node.

Structural hashing is most powerful when combined with algebraic simplification. This involves rewriting the DAG based on algebraic equalities. Each rewrite simplifies the representation, but also potentially helps uncover additional shared structure as illustrated in the figure. The current Sketch solver release has a hand-crafted simplifier, although in recent work, we have explored

the automatic synthesis of the simplification layer [1].

The workhorse for solving constraints is the SAT solver, which is able to take a boolean formula expressed in Conjunctive Normal Form (CNF) and either generate a satisfying assignment or prove that the constraints are unsatisfiable, i.e. that they have no solution. Over the rest of this section, we describe how the constraints from the previous lecture are translated into boolean constraints in CNF form, and how the SAT solver is able to solve them.

8.3 From high-level constraints to CNF



In the boolean satisfiability literature, a *Literal* is either a variable or its negation. A *Clause* is a disjunction (or) of literals. A formula is said to be in Conjunctive Normal Form if it consists of a conjunction (and) of clauses.

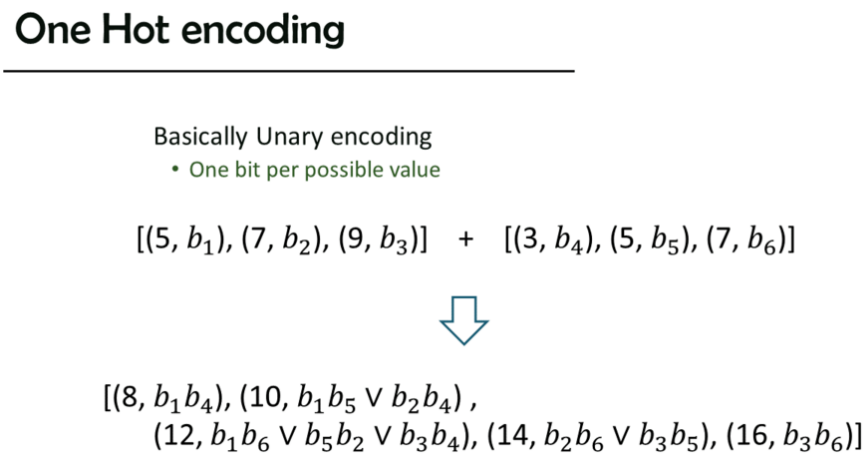
The CNF representation has a number of advantages. A particularly important one is that we can turn an arbitrary boolean formula into CNF format in polynomial time. This is unlike Disjunctive Normal Form (DNF) which may require exponential time to generate.

The basic approach for generating a CNF formula from an arbitrary boolean formula is illustrated by the figure. The approach is based on the observation that a formula of the form $\wedge_i l_i \Rightarrow l_j$ is trivially converted into a CNF formula. Given a boolean formula represented as a DAG, it is easy to define for each node a set of implications that relate the values of the input to the values of the output. For example, given a node of the form $t1 = h_0 \wedge h_1$, we can write out the set of implications that define the relationship between $t1$, h_0 and h_1, namely,

$$\begin{aligned} h_0 \wedge h_1 \Rightarrow t_1 &\equiv \bar{h_0} \vee \bar{h_1} \vee t_1 \\ t_1 \Rightarrow h_0 &\equiv \bar{t_1} \vee h_0 \\ t_1 \Rightarrow h_1 &\equiv \bar{t_1} \vee h_1 \end{aligned}$$

In addition to Booleans, though, Sketch supports Integers, Floating point values, bit-vectors, arrays and recursive datatypes. Most of these types are supported through *one-hot* encodings.

8.4 One-hot encoding in Sketch



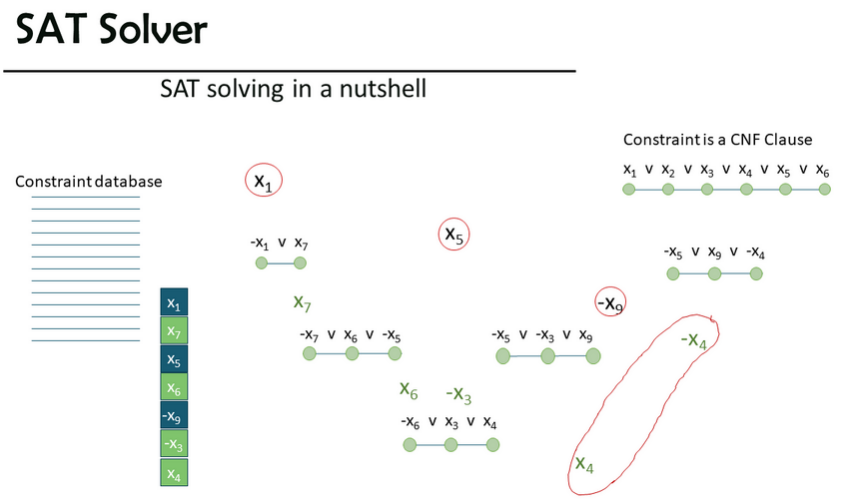
A One-hot encoding is essentially a unary encoding. The basic idea is to have a separate indicator variable for each possible value of a variable that indicates what the true value is. For example, the figure illustrates a one-hot encoding where two variables, each with three possible values are added together. Each value is represented as a list of value, indicator variable pairs. The result contains all possible values that can result from adding the original two numbers, and the new indicator variables are now boolean combinations of the indicator variables in the original values.

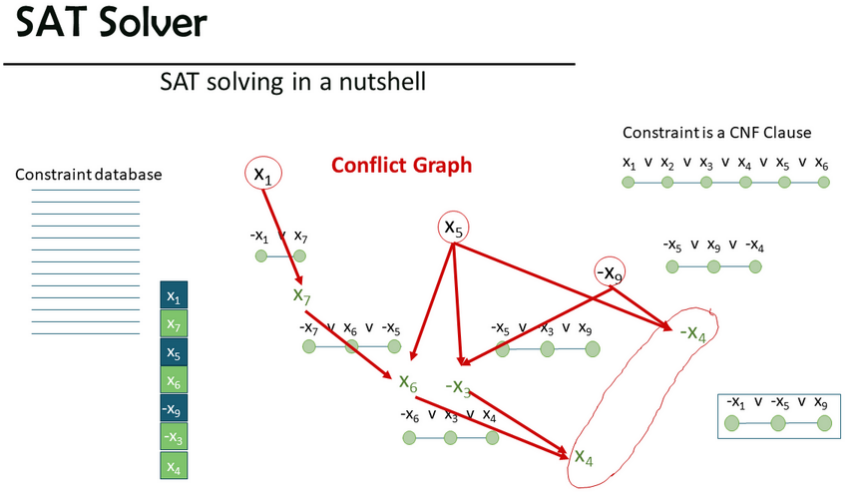
By default, every integer in sketch is represented using this one-hot encoding, and when using the "`-fe-fpencoding TO_BACKEND`" flag, even floating point values are stored using this encoding as well. By default, even arrays are represented using this encoding. Every entry in the array having one bit for every possible value for that entry. Recursive data-types are also represented using this kind of encoding; the details are beyond the scope of this lecture, but can be found on a recent paper by Inala et al [2].

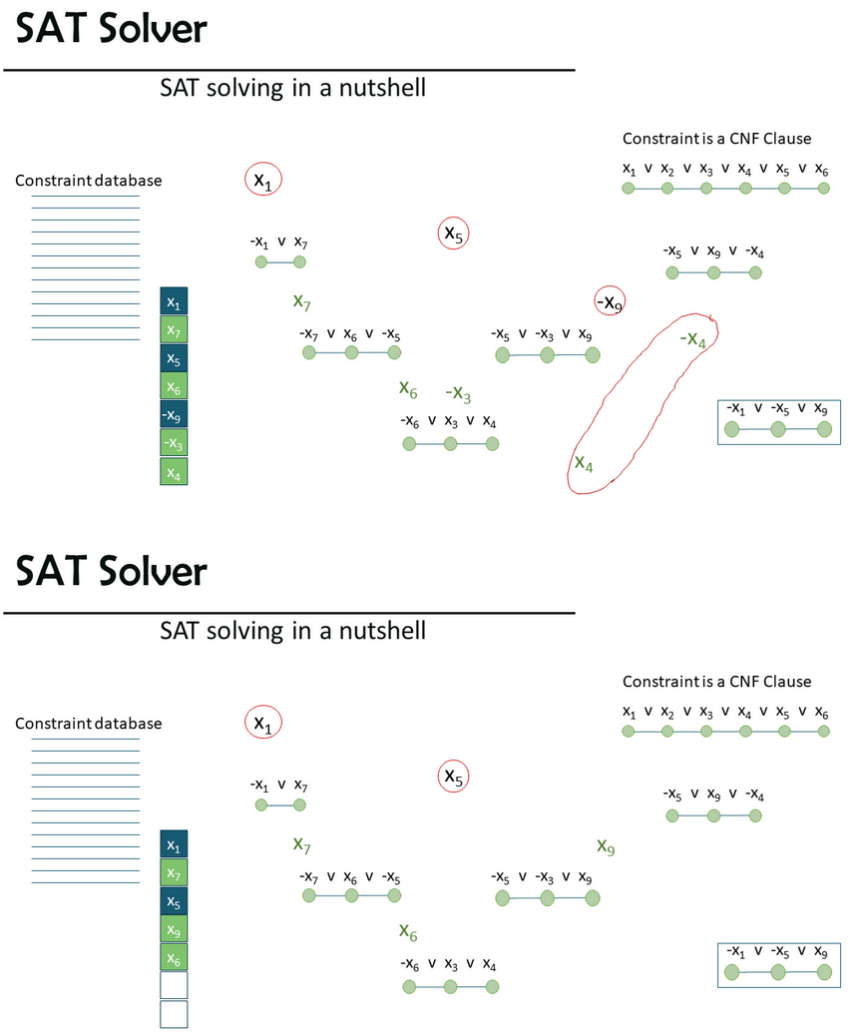
There are a number of advantages to this encoding, particularly its simplicity and flexibility. Most importantly, the encoding can also be extremely efficient in allowing the SAT solver to propagate information about the possible values of a variable. The major downside of this encoding is that it is very space inefficient. As the number of possible values for a variable grows, so does the number of bits required to represent it. This can be particularly problematic

for sketches that are heavy in arithmetic. For those sketches, sketch provides an alternative solver that does not rely on one-hot encodings, and which can be enabled with the flag "`-slv-nativeints`".

8.5 Solving SAT problems







Modern SAT solvers are based on the DPLL algorithm named after Martin Davis, George Logemann, Donald Loveland and Hilary Putnam. The algorithm performs a backtracking search over the space of possible assignments to a boolean formula, but the key idea of the algorithm is that every time it makes a choice for the value of a variable, it propagates the logical implications of that assignment. The process is illustrated in the figure. After the variable x_1 is set, the clause $\bar{x_1} \vee x_7$ forces the value of x_7 to be true through *unit propagation*. As the assignment of variables and propagation of logical consequences continues, eventually one of two things happen, either the solver assignes values to all the variables and terminates, or it runs into a contradiction as illustrated in the figure where the assignment of false to x_9 implies that x_4 is both true and false according to different clauses.

Modern SAT solvers improve on this basic algorithm in three important ways. The first one, also illustrated in the figure is *conflict analysis*. Every time the solver arrives at a contradiction, it traces back to identify a small set of assignments that led to that contradiction. In the case of the example, we can lay the blame of the conflict on the assignment to x_1, x_5, and $\bar{x_9}$. This explanation for the conflict is summarized in a *conflict clause* that prevents that bad assignment from appearing again. The conflict clause is not unique; for example, $\bar{x_7} \vee \bar{x_5} \vee x_9$ would also be an acceptable conflict, because as we can see from the figure, an assignment with x_7, x_5 and $\bar{x_9}$ would also lead to the same contradiction we observed.

More formally, every time we observe a conflict, we can define a *conflict graph* where each node N corresponds to a variable, and there is a directed edge (n_1, n_2) connecting two nodes iff the clause that forced n_2 to be set to a value includes variable n_1. It is not hard to see that this conflict graph will be a DAG. The source nodes will be the variables that were chosen arbitrarily, and the inner nodes correspond to the assignments that were implied by those arbitrary assignments. Every time a contradiction is reached, the conflict graph can tell us what the possible conflict clauses are. Specifically, any set of nodes in the graph that separates the decision variables (the sources in the DAG) from the conflict will make a valid conflict clause. Thus, in the example, the conflict graph tells us that another valid conflict clause would be $\bar{x_6} \vee x_3 \vee x_9$. This process of learning conflict clauses is termed *Conflict Driven Clause Learning* (CDCL) and was first proposed by Marques Silva and Sakallah in their seminal paper on the GRASP SAT Solver [3].

The second important improvement over the basic algorithm in addition to CDCL is called *two literal watching*, which was first developed by Moskewicz, Madigan, Zhao, Zhang and Malik in the Chaff SAT solver [4]. The key observation behind two-literal watching is that while in principle, every time we set the value of a variable we have to visit all the clauses that include that variable, in practice, the only cases where a clause actually leads to some action is when we set the second to last literal. At that point, if all other literals have been set to false, and this second to last literal is also set to false, then the last remaining literal must be set to true for the clause to be satisfied, and we get unit propagation. So unit propagation will only happen when we set the second to last variable. So the idea is for every clause, we keep track of two literals that haven't been set. As long as there are two literals that have not been set, setting other literals in the clause will have no effect, so there is no need to do anything. Only when one of the two unwatched literals is set, then we check one of three possibilities: (a) maybe some of the other literals we were not watching was already set to true, in which case the clause is already satisfied and there is nothing else to do; (b) maybe there are other literals that have not been set, in which case we just switch the literals we currently watch so that we are again

watching to unassigned literals, or (c) these two literals are the last unassigned literals and all others have been set to false, in which case we do unit propagation. By watching only two literals at a time, the solver saves an enormous amount of memory trafic by only having to visit a small fraction of the clauses every time an variable is assigned.

Finally, the third important optimization in a modern SAT solver involves being careful (but not too careful) in picking which variables to assign next. The point about not being too careful is actually important. The most popular heuristic is Variable State Independent Decaying Sum (VSIDS), which is very simple, but also very fast, so even though it is not too sophisticated, because it is very fast, it allows us to search more efficiently overall. The basic idea in VSIDS is to keep a score for every variable that is additively bumped based on how much that variable is used, for example in conflict clauses, and then it is exponentially decayed over time.

Overall, the combination of clever heuristics and careful engineering allows modern SAT solvers to solve synthesis problems with millions of variables and clauses. Later in the course we will discuss SMT solvers, which are built on top of SAT solvers and provide additional expressive power that is particularly important for dealing with verification problems.

References

[1] Rohit Singh and Armando Solar-Lezama. SWAPPER: A framework for automatic generation of formula simplifiers based on conditional rewrite rules. In *2016 Formal Methods in Computer-Aided Design, FMCAD 2016, Mountain View, CA, USA, October 3-6, 2016*, pages 185–192, 2016. doi: 10.1109/FMCAD.2016.7886678. URL https://doi.org/10.1109/FMCAD.2016.7886678.

[2] Jeevana Priya Inala, Nadia Polikarpova, Xiaokang Qiu, Benjamin S. Lerner, and Armando Solar-Lezama. Synthesis of recursive ADT transformations from reusable templates. In *Tools and Algorithms for the Construction and Analysis of Systems - 23rd International Conference, TACAS 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22-29, 2017, Proceedings, Part I*, pages 247–263, 2017. doi: 10.1007/978-3-662-54577-5_14. URL https://doi.org/10.1007/978-3-662-54577-5_14.

[3] João P. Marques Silva and Karem A. Sakallah. GRASP - a new search algorithm for satisfiability. In *ICCAD*, pages 220–227, 1996. doi: 10.1109/ICCAD.1996.569607. URL https://doi.org/10.1109/ICCAD.1996.569607.

[4] Matthew W. Moskewicz, Conor F. Madigan, Ying Zhao, Lintao Zhang, and Sharad Malik. Chaff: Engineering an efficient SAT solver. In *Proceedings of the 38th Design Automation Conference, DAC 2001, Las Vegas, NV, USA, June 18-22, 2001*, pages 530–535, 2001. doi: 10.1145/378239.379017. URL http://doi.acm.org/10.1145/378239.379017.